

Data Structure

UNIT 1: DATA STRUCTURES & ALGORITHM ANALYSIS

1. Introduction to Data Structures

A data structure is a specialized format for organizing, processing, retrieving, and storing data. They are designed to fit specific purposes so that data can be accessed and worked with in efficient ways.

2. Classification

- **Primitive Data Structures:** Basic types like Integer, Float, Character, and Boolean.
- **Non-Primitive Data Structures:**
 - * **Linear:** Data elements are arranged in a sequential manner (Arrays, Linked Lists, Stacks, Queues).
 - **Non-Linear:** Data elements are arranged in a hierarchical manner (Trees, Graphs).

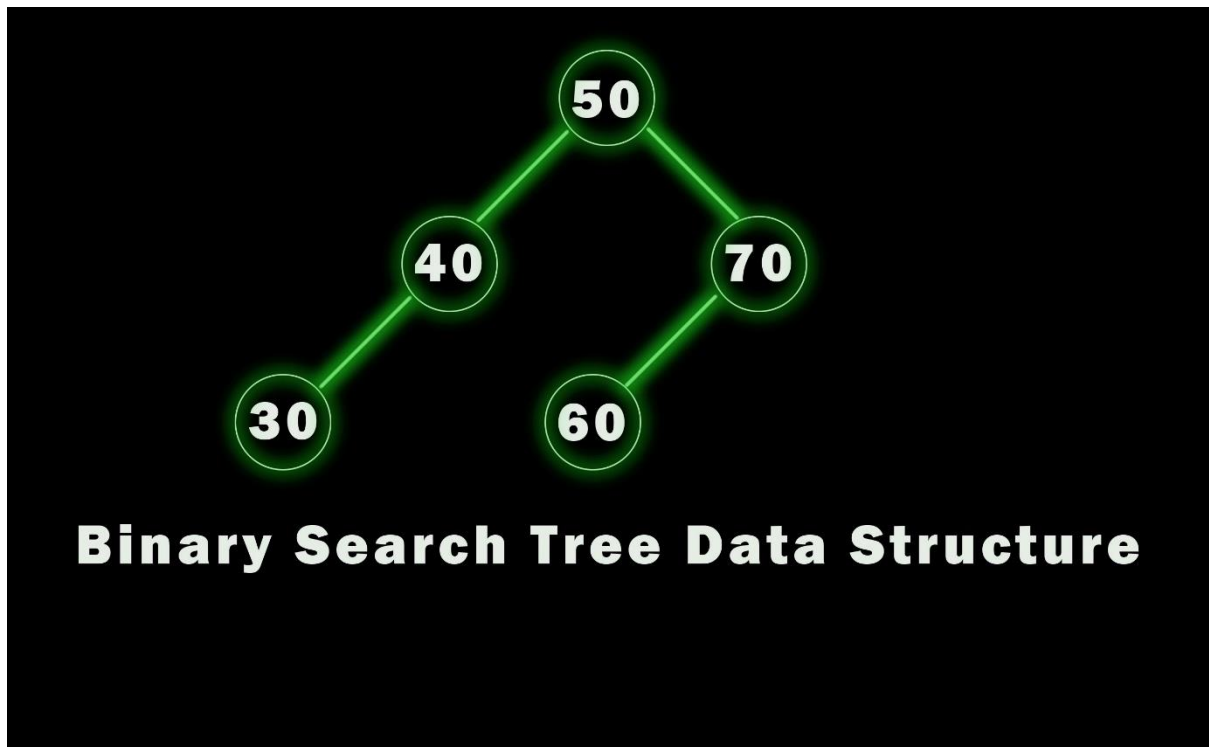
3. Linear Data Structures in Detail

- **Arrays:** A collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together.
 - *Advantage:* $O(1)$ time complexity for accessing elements by index.
- **Linked Lists:** Unlike arrays, elements are not stored in contiguous locations; they are linked using pointers.
 - *Types:* Singly Linked List, Doubly Linked List, and Circular Linked List.
- **Stacks (LIFO):** A linear data structure that follows the **Last-In-First-Out** principle.
 - *Operations:* Push() (add), Pop() (remove), Peek() (view top).
 - *Use Cases:* Function calls, Undo mechanisms in editors.
- **Queues (FIFO):** A linear data structure that follows the **First-In-First-Out** principle.
 - *Operations:* Enqueue() (add), Dequeue() (remove).
 - *Use Cases:* CPU scheduling, Printer task management.

4. Non-Linear Data Structures

- **Trees:** A hierarchical structure with a root value and subtrees of children with a parent node.
 - **Binary Search Tree (BST):** A tree where the left child is smaller than the parent and the right child is larger.
- **Graphs:** A collection of nodes (vertices) and the connections between them (edges). Graphs are used to represent networks like social media or GPS paths.

Data Structure



Shutterstock

Explore

5. Algorithm Complexity (Big O Notation)

To measure the efficiency of an algorithm, we use:

- **Time Complexity:** The amount of time an algorithm takes to run as a function of the input size n .
- **Space Complexity:** The amount of memory space required by an algorithm.
- *Common Notations:* $O(1)$ (Constant), $O(n)$ (Linear), $O(n^2)$ (Quadratic), $O(\log n)$ (Logarithmic).